

Week 6. Multi-Networking & Secondary Networking

Multus Architecture

- Multus는 Pod에 여러 네트워크 인터페이스를 붙여주는 CNI 플러그인이다
- 직접 인터페이스를 만들지 않고 다른 CNI를 호출하는 **meta-plugin**으로 동작함
- 기본 Pod는 loopback을 제외하면 eth0 하나뿐이지만, Multus를 쓰면 인터페이스가 여러 개인 multi-homed Pod를 만들 수 있음
- 추가 네트워크 정의는 Network Plumbing WG 표준 (NetworkAttachmentDefinition)을 따름

인터페이스	담당	역할
eth0	default network (Flannel, Calico 등)	클러스터 통신, Pod IP, Service
net1, net2 ...	추가 네트워크 (macvlan, ipvlan, SR-IOV 등)	트래픽 분리, 하드웨어 가속

구성 요소

- **Multus CNI 설정:** `/etc/cni/net.d/00-multus.conf`. 런타임은 사전순 첫 번째 설정 파일을 사용하므로, Multus가 먼저 호출되도록 00을 붙임
- **kubeconfig:** NAD를 조회하기 위해 API Server에 접근할 때 사용함
- **NetworkAttachmentDefinition (NAD):** 추가 네트워크 하나의 CNI 설정을 담은 CRD이다
- **delegate CNI:** Multus가 호출해 실제 인터페이스를 만드는 플러그인이다
 - Main(macvlan 등)이 인터페이스를 만들고 IPAM(whereabouts 등)이 IP를 할당함

Default Network 지정

- Multus 설정에서 default network를 반드시 지정해야 함
 - `clusterNetwork` : default CNI를 지정함. `defaultNetworks` 로 모든 Pod에 기본으로 붙일 추가 네트워크도 지정 가능함

- `delegates` : CNI 설정 목록을 직접 넣음. 첫 번째 항목이 Pod IP를 담당함
- 둘은 함께 쓸 수 없음
- `readinessindicatorfile` 로 default network가 준비된 뒤에 추가 네트워크를 붙이도록 할 수 있음

Thin Plugin vs Thick Plugin

구분	Thin	Thick (4.0부터, 권장)
구조	multus 바이너리 하나가 모두 처리	shim(클라이언트) + daemon(서버)
통신	없음	unix socket 위에서 REST 요청
장점	가볍고 단순함	메트릭 등 추가 기능
단점	기능이 적음	리소스를 더 사용함

```
runtime → shim → (unix socket) → daemon → delegate CNI
```

- thick에서는 daemon이 NAD 조회, RuntimeConfig 계산, delegate 호출을 모두 맡음
- daemon이 Pod 안에서 delegate를 실행하므로 호스트 파일이 필요한 CNI는 chrootDir 옵션이나 볼륨 마운트가 필요함

Pod 생성 흐름

1. CRI가 Multus(thick이면 shim)를 CNI ADD로 호출함
2. Multus가 default network를 호출해 eth0을 만듦
3. defaultNetworks 설정과 Pod annotation `k8s.v1.cni.cncf.io/networks` 를 보고 NAD를 조회함
4. NAD에 정의된 CNI를 순서대로 호출해 net1, net2 ... 를 만듦
5. 결과를 런타임에 반환함

CNI Chaining

- 하나의 네트워크 설정 안에서 여러 CNI 플러그인을 순서대로 실행하는 CNI 스펙 기능이다
- 앞 플러그인이 만든 인터페이스에 뒤 플러그인이 기능을 덧붙이는 방식이다

- 인터페이스 수는 늘지 않고, 같은 인터페이스에 기능이 쌓임

동작 방식

- `.conflist` 파일의 `plugins` 배열에 플러그인을 나열함
- ADD는 배열 순서대로, DEL은 역순으로 실행됨
- 앞 플러그인의 실행 결과(`prevResult`)가 다음 플러그인의 입력으로 전달됨
- 보통 첫 번째는 인터페이스를 만드는 Main 플러그인, 그 뒤는 기능을 추가하는 Meta 플러그인임

```
{
  "cniVersion": "0.4.0",
  "name": "k8s-pod-network",
  "plugins": [
    { "type": "calico" },
    { "type": "portmap", "capabilities": { "portMappings": true } },
    { "type": "bandwidth", "capabilities": { "bandwidth": true } }
  ]
}
```

calico (eth0 생성, IP 할당)
 → portmap (hostPort용 iptables 규칙 추가)
 → bandwidth (eth0에 대역폭 제한 추가)

대표적인 Meta 플러그인

플러그인	역할
portmap	hostPort 포트 매핑
bandwidth	대역폭 제한
tuning	sysctl, MTU 등 인터페이스 설정 변경
firewall	방화벽 규칙 추가
sbr	source based routing

Cilium CNI Chaining

- CNI Chaining을 활용해 Cilium을 다른 CNI 플러그인과 함께 쓰는 모드이다
- 역할 분담
 - 다른 CNI 플러그인: 기본 네트워크 연결성과 IP 주소 관리(IPAM) 담당
 - Cilium: 그 플러그인이 만든 네트워크 장치에 eBPF 프로그램을 붙여 기능 추가
- Cilium이 제공하는 기능
 - L3/L4 네트워크 가시성
 - 정책 강제(policy enforcement)
 - 그 밖의 고급 기능
- 활용 상황
 - AWS VPC CNI처럼 클라우드 네트워크와 직접 연동되는 CNI는 유지하면서 Cilium의 정책과 가시성만 얹고 싶을 때 사용함

AWS VPC CNI (인터페이스 생성, VPC IP 할당)
→ cilium-cni (생성된 장치에 eBPF 프로그램 부착)

지원 대상

대상	비고
AWS VPC CNI	EKS 기본 CNI
Azure CNI (Legacy)	
Calico	
Generic Veth Chaining	veth 기반 CNI 전반에 적용하는 범용 방식
OKE VCN-Native Pod 네트워킹	Oracle Kubernetes Engine
Portmap (HostPort)	HostPort 지원용
Weave Net	

Multus와의 차이

구분	CNI Chaining	Multus
정체	CNI 스펙 기능	meta-plugin
결과	인터페이스 하나에 기능 추가	인터페이스 여러 개 추가

구분	CNI Chaining	Multus
방향	세로로 쌓음	가로로 늘림
설정	conflist의 plugins 배열	NAD + Pod annotation

- 둘은 함께 쓸 수 있음. Multus가 호출하는 delegate나 NAD 설정 자체가 conflist(체인)일 수 있음
- Multus의 `auxiliaryCNIChainName` 은 default network와 분리된 별도 체인을 모든 Pod에 실행하는 기능임

Primary / Secondary Network

- Pod에 붙는 네트워크는 역할에 따라 primary와 secondary로 나뉨
- 핵심은 **Kubernetes는 primary network만 인지한다**는 점이다

구분	Primary Network	Secondary Network
인터페이스	eth0	net1, net2 ...
담당 CNI	Flannel, Calico, Cilium 등	macvlan, ipvlan, SR-IOV 등
Pod IP (<code>status.podIP</code>)	O	X, annotation에만 기록
Service Endpoint / DNS 레코드	등록됨	등록 안 됨
NetworkPolicy	적용됨	적용 안 됨 (MultiNetworkPolicy 별도 필요)

멀티 네트워크가 필요한 상황

- 서비스, 관리, 스토리지 트래픽 분리
- 5G, NFV처럼 control plane과 data plane을 다른 망으로 나누는 워크로드
- SR-IOV, DPDK로 커널을 우회하는 고성능 데이터 플레인
- 기존 L2 망(VLAN)에 Pod를 직접 연결해야 할 때
- 테넌트별 보안 격리

NetworkAttachmentDefinition (NAD)

- secondary network 하나를 정의하는 CRD이다 (`k8s.cni.cncf.io/v1`)

- 내용물은 사실상 CNI 설정 JSON이다
- Namespace 범위 리소스이며 shortName은 `net-attach-def`

구조

```
apiVersion: k8s.cni.cncf.io/v1
kind: NetworkAttachmentDefinition
metadata:
  name: macvlan-conf
spec:
  config: |
    {
      "cniVersion": "0.3.1",
      "type": "macvlan",
      "master": "eth1",
      "mode": "bridge",
      "ipam": { "type": "whereabouts", "range": "192.168.10
0.0/24" }
    }
```

- `type`: 인터페이스를 만들 CNI 플러그인
- `master`: 부모가 될 노드의 NIC
- `ipam`: IP 할당 방식

IPAM 선택

IPAM	범위	특징
host-local	노드 단위	노드 간 IP 할당 상태를 공유하지 않아 충돌 가능
whereabouts	클러스터 단위	클러스터 전체에서 관리해 충돌 방지
static	Pod 단위	IP를 직접 지정
dhcp	외부 DHCP	노드에 dhcp 데몬 필요

- secondary network는 노드 구분 없이 하나의 서브넷을 쓰는 경우가 많아 whereabouts 같은 클러스터 범위 IPAM이 적합함

Pod에서 참조

- annotation `k8s.v1.cni.cncf.io/networks` 로 NAD를 지정함
- `name`, `namespace/name`, `name@인터페이스명` 형식이나 JSON 형식을 쓸 수 있음

macvlan / ipvlan / SR-IOV

macvlan

- 부모 NIC 위에 **고유한 MAC을 가진** 서브 인터페이스를 만듦
- 외부에서는 Pod가 독립된 L2 호스트처럼 보임
- 주요 모드는 bridge(같은 부모를 쓰는 서브 인터페이스끼리 직접 통신)
- 제약
 - 호스트는 부모 인터페이스를 통해 macvlan 인터페이스와 직접 통신할 수 없음
 - MAC이 Pod 수만큼 늘어 스위치 MAC 제한에 걸릴 수 있음
 - 퍼블릭 클라우드는 등록되지 않은 MAC을 버리는 경우가 많음
- 적합한 사례: 온프레미스에서 기존 VLAN에 Pod를 직접 연결할 때

ipvlan

- 서브 인터페이스들이 **부모 NIC의 MAC을 공유**하고 IP만 다름
- 모드
 - I2: macvlan bridge와 유사
 - I3: 부모가 라우터처럼 L3 기준으로 포워딩
 - I3s: I3에 호스트 netfilter(conntrack) 적용이 추가된 모드
- 제약
 - 같은 MAC을 쓰므로 MAC 기반 DHCP가 어려움
 - 호스트와의 직접 통신에 제약이 있음
- 적합한 사례: MAC 수 제한이 있는 환경, L3 라우팅 기반 구성

SR-IOV

- **NIC 하드웨어를 가상으로 쪼개는** 기술이다
 - PF (Physical Function): 실제 물리 NIC

- VF (Virtual Function): PF에서 나온 가상 NIC
- VF를 Pod에 직접 붙여 호스트 쪽 bridge, veth, iptables를 거치지 않음
- 구성 요소
 - SR-IOV 지원 NIC
 - SR-IOV Network Device Plugin: VF를 노드 리소스로 등록함
 - SR-IOV CNI: 할당된 VF를 Pod에 붙임
- Pod는 CPU, 메모리처럼 resources로 VF를 요청함
- 제약: 하드웨어 의존적이고 VF 개수가 제한됨
- 적합한 사례: NFV, 5G 등 처리량과 지연이 핵심인 워크로드

비교

항목	macvlan	ipvlan	SR-IOV
구현	커널	커널	NIC 하드웨어
MAC	인터페이스마다 고유	부모와 공유	VF마다 고유
성능	높음	높음	매우 높음
하드웨어 요구	없음	없음	SR-IOV 지원 NIC
대표 사용처	기존 L2/VLAN 연결	MAC 제한 환경	NFV, 초저지연

Multi-NIC Pod 구성 및 Packet Flow

구성

- NAD를 만들고 Pod annotation으로 연결하면 eth0 외에 net1, net2가 붙음
- secondary IP는 Pod annotation `k8s.v1.cni.cncf.io/network-status` 에서 확인함

```
metadata:
  annotations:
    k8s.v1.cni.cncf.io/networks: macvlan-conf, ipvlan-conf
```

인터페이스 선택

- 인터페이스가 여러 개여도 어느 인터페이스로 나갈지는 Pod 내부 라우팅 테이블이 결정함

```
default via 10.244.1.1 dev eth0          # primary
10.244.1.0/24 dev eth0                   # primary
192.168.100.0/24 dev net1                 # secondary
```

- secondary 서브넷으로 가는 트래픽 → net1
- 그 외 모든 트래픽 (다른 Pod, Service, 인터넷) → default route를 따라 eth0
- secondary 너머의 다른 대역을 쓰려면 NAD의 IPAM에 route를 추가해야 함
- 들어온 인터페이스와 나가는 인터페이스가 다르면 비대칭 라우팅으로 패킷이 버려질 수 있음

인터페이스별 경로

인터페이스	경로	특징
eth0 (primary)	Pod → veth → 호스트 netns → iptables/eBPF → 라우팅/overlay → NIC	Service, NetworkPolicy 적용
net1 (macvlan/ipvlan)	Pod → 부모 NIC → 외부 스위치	호스트 iptables를 거치지 않음 (ipvlan I3s 제외)
net2 (SR-IOV)	Pod → VF → NIC 내부 스위치 → 외부	호스트 쪽 커널 경로를 거치지 않음

- secondary 경로는 호스트 iptables를 거치지 않아 빠름
- Service와 NetworkPolicy가 적용되지 않는 근본 이유는 Kubernetes가 secondary IP를 모르기 때문이다
 - Service Endpoint에는 primary IP만 등록됨
 - NetworkPolicy 구현체도 primary 인터페이스만 대상으로 동작함

▼ 참고자료

<https://github.com/k8snetworkplumbingwg/multus-cni>

<https://k8snetworkplumbingwg.github.io/multus-cni/docs/thick-plugin.html>

<https://k8snetworkplumbingwg.github.io/multus-cni/docs/configuration.html>

